Test Case Document v2.0: File Compression Tool

Project: File Compression Tool (Basic ZIP Implementation): Team 14
Team: Pranavi Sistla (PES1UG24CS338), Pavan Kumar H K (PES1UG24CS319), Rachit Upadhyay (PES1UG24CS355), Inchara K Kuppal (PES1UG25CS818)
Based on: Software Test Plan (STP) v1.1, Section 13; SRS v1.1; SAD v1.0; `Documents/DEVIATIONS.md`
Date: 05-Nov-2026
Status: Final (Part-3 deliverable 1: Test Cases Updation)

Version	Date	Change
1.0	06-Oct-2026	14 test cases in STP v1.0, Section 13
1.1	28-Oct-2026	STP v1.1: roles, SRS 6.x references and schedule corrected (no test case changes)
2.0	**05-Nov-2026**	**Test cases updated for the changes made during Sprint 1, Sprint 2 and Sprint 3: concrete commands and expected values, 10 new system test cases, 36 unit test cases, actual results recorded**

---

1. Why the test cases changed

The original 14 test cases were written before any code existed, so their steps and expected results were generic (for example "tree and encoded data are generated correctly"). During the sprints the design became concrete and several defects were found and fixed. Each change below updated or added test cases.

Change during the sprints	Source	Effect on test cases
CLI syntax fixed: `-c`, `-d`, `-o`, `-h`; exit codes 0/1/2/3	PR #15 (REQ-1)	All system tests now use exact commands and check exit codes. New **TC-ERR-06** (wrong usage), **TC-ERR-07** (folder as input)

Change during the sprints	Source	Effect on test cases
`.huff` layout fixed: 8-byte size + 2048-byte frequency table + 8-byte bit count + payload	PR #16, `FORMAT.md`, D-05	**TC-COMP-02/03/04** now check exact header bytes from the FORMAT.md worked example
Deterministic Huffman tie-breaking (cross-OS bug)	PR #24, D-09	**TC-COMP-03** expects one exact payload (`69 CF 68`). **TC-NF-03** checks Mac ↔ Windows archives
Compression metrics added	PR #17, D-08	**TC-COMP-06** checks the three labels in the output
Default output names (`<input>.huff`, `<name>.out`)	PR #15, #25	New **TC-DEC-04**, **TC-DEC-05**
Output may not overwrite the input; errors exit cleanly	PR #23, D-14	New **TC-ERR-05**
Writes go to a temp file, then rename	PR #31, D-14	**TC-ERR-02** also checks that an existing output is untouched. New **TC-ERR-09**
Streaming I/O reduced memory	PR #31, D-10	**TC-NF-02** re-measured
Trailing data and single-symbol payload checks restored	PR #33, D-17	New **TC-ERR-03**, **TC-ERR-04**
Empty-file edge case (SAD 4.6)	SAD 4.6	New **TC-COMP-07**
Corrupted payload bits are not always detected	D-16 (known limitation)	**TC-SEC-01** records the expected behaviour (not counted as a failure)
Error-path unit tests added in Sprint 3	Part-3 testing	**UT-ROB-01 to UT-ROB-18** (Section 4)

---

2. Test levels and how they are run

Level	What	Command	Automated in CI
Unit	36 unit tests (`tests/test_*.cpp`)	`make test`	✅ every push / PR, on Ubuntu, Windows, macOS
System	21 system test cases (Section 3)	`bash tests/system_validation.sh`	✅
System	17 lossless/corruption checks (issue #11)	`bash tests/lossless_check.sh`	✅
Non-functional	TC-NF-01/02/03	see Section 3.4	Partly (portability via the CI matrix)

Level	What	Command	Automated in CI
Coverage	Line, function and branch coverage	`bash tests/coverage.sh` (needs `gcovr`)	Run manually

Result codes: PASS = expected result observed · FAIL = not observed · KL = known limitation (documented in DEVIATIONS, not a failure)

---

3. System test cases

All actual results below are from the run on **05-Nov-2026** (`main` @ `cae72ce` + the Sprint-3 tests). See `System_Validation_Report.md`.

3.1 Compression (REQ-1 to REQ-4)

ID	Req.	Test case	Procedure	Expected result	Actual result	Status
TC-COMP-01	REQ-1	Valid file path	`huffzip -c abra.txt -o abra.huff`	Exit 0, `abra.huff` created	Exit 0, file created	PASS
TC-COMP-02	REQ-2	Frequency calculation	Compress `abracadabra`; read the frequency table at offsets 784 (`a`) and 920 (`r`)	`a` = 5, `r` = 2 (little-endian uint64)	`05 00…`, `02 00…`	PASS
TC-COMP-03	REQ-3	Huffman tree and encoding	Same archive; read the bit count (offset 2056) and payload (offset 2064)	23 bits; payload `69 CF 68` (FORMAT.md §7)	23 bits; `69 CF 68`	PASS
TC-COMP-04	REQ-4	Compressed output in the custom format	`huffzip -c numbers.txt` (no `-o`); check the `abra.huff` size and header	`numbers.txt.huff` created; `abra.huff` = 2067 bytes (2064 header + 3); size field = 11	As expected	PASS
TC-COMP-05	REQ-3	Single-symbol file	Compress and decompress 20 × `A`; `cmp`	Restored file identical	Identical	PASS
TC-COMP-06	SRS 2.2	Compression metrics	`huffzip -c numbers.txt -o metrics.huff`	Output shows `Original size:`, `Compressed size:`, `Ratio:`	All three shown	PASS
TC-COMP-07	SAD 4.6	Empty file *(new)*	Compress and decompress a 0-byte file	Archive = 2064 bytes (header only); restored = 0 bytes	As expected	PASS

3.2 Decompression (REQ-5 to REQ-7)

ID	Req.	Test case	Procedure	Expected result	Actual result	Status
TC-DEC-01	REQ-5	Metadata extraction	`huffzip -d abra.huff -o abra.out`	Reports `Decompressed size: 11 bytes`	As expected	PASS
TC-DEC-02	REQ-6	Bitstream decoding	`cmp abra.txt abra.out`	Identical (`abracadabra`)	Identical	PASS
TC-DEC-03	REQ-7	Lossless restoration	Round trip of 1.3 MB text, 100 KB random binary and an all-byte file; `cmp` each	All three identical	All identical	PASS
TC-DEC-04	REQ-7	Default output name *(new)*	`huffzip -d abra.huff` (no `-o`)	`abra.out` written and identical	As expected	PASS
TC-DEC-05	REQ-7	Archive without `.huff` extension *(new)*	`huffzip -d archive.bin`	`archive.bin.out` written and identical	As expected	PASS

3.3 Error handling and safety (SRS 6.2, SAD 3.9 / 4.4, STP 5.1)

ID	Req.	Test case	Procedure	Expected result	Actual result	Status
TC-ERR-01	REQ-1 / SAD 4.4	Invalid path	`huffzip -c does_not_exist.txt -o x.huff`	"not found" message, exit 2, no `x.huff`	As expected	PASS
TC-ERR-02	SRS 6.2	Corrupted (truncated) archive	Cut 40 bytes off a valid archive; decompress onto an existing file `keep.out`	Non-zero exit; `keep.out` unchanged; no `.tmp` left	As expected	PASS
TC-ERR-03	SRS 6.2 / D-17	Trailing data *(new)*	Append `XYZ` to a valid archive; decompress	Rejected, no output	Rejected	PASS
TC-ERR-04	SRS 6.2 / D-17	Single-symbol archive without payload *(new)*	Keep only the 2064-byte header of a single-symbol archive	Rejected, no output	Rejected	PASS
TC-ERR-05	SRS 6.2 / D-14	Output equal to input *(new)*	`huffzip -c same.txt -o same.txt`	"must be different" message, exit 1, input unchanged	As expected	PASS
TC-ERR-06	SRS 3.1	Wrong usage *(new)*	No arguments; `-x`; `-q` instead of `-o`; then `-h`	Exit 1 for the three errors; `-h` prints usage, exit 0	Codes 1,1,1,0	PASS

ID	Req.	Test case	Procedure	Expected result	Actual result	Status
TC-ERR-07	REQ-1	Folder as input *(new)*	`huffzip -c adir -o d.huff`	Exit 2, no output	As expected	PASS
TC-ERR-08	SRS 6.2	Not an archive *(new)*	Decompress a plain text file	Rejected, no output	Rejected	PASS
TC-ERR-09	D-14	Replace an existing output safely *(new)*	Compress onto an existing `existing.huff`, then decompress it	Replaced via temp file + rename; round trip identical	As expected	PASS
TC-SEC-01	SRS 6.2 / D-16	Single bit flip inside the payload *(new)*	Flip one payload bit in 200 random positions	Structural damage rejected; **some flips decode to wrong data without an error** (no checksum)	60/200 rejected, 140/200 not detected	KL

3.4 Non-functional (SRS 6.1, 6.4, 2.4)

ID	Req.	Test case	Procedure	Expected result	Actual result	Status
TC-NF-01	SRS 6.1	100 MB performance	Compress and decompress a 100 MiB file, 3 runs (`TC-NF-01_Performance_Results.md`)	Each operation under 10 s	Windows i5-12450H: **4.35 s / 0.57 s**. Linux re-check 05-Nov (103 MB): 1.9–2.2 s / 3.5–3.6 s. Restored identical	PASS
TC-NF-02	SRS 6.1	Memory usage	Peak RSS while processing a 103 MB text file	At most file size + tree overhead	Compress **101 MB** (about 1.0×), decompress **11 MB**	PASS
TC-NF-03	SRS 2.4 / 6.4	Portability	CI build and tests on 3 OSes; Mac ↔ Windows archive exchange (`PORTABILITY.md`)	Builds and tests pass on all; archives byte-identical across OSes	All green; `fc /b` and `cmp` identical both ways	PASS
TC-NF-04	SRS 6.4	Lossless correctness *(issue #11)*	`bash tests/lossless_check.sh`: 12 round trips + 5 corruption cases	17/17 pass	17/17	PASS

---

4. Unit test cases (36)

Run with `make test`. The full results are in `Unit_Test_Report.md`.

Group (file, owner)	IDs	Count	What they cover
Huffman (`tests/test_huffman.cpp`, Pavan)	UT-HUF-01 to 07	7	Empty input, single symbol, basic codes, deterministic tie-breaking, binary symbols, packed size, missing code
Input and frequencies (`tests/test_input.cpp`, Pranavi)	UT-IN-01 to 04	4	Valid path (TC-COMP-01), invalid path (TC-ERR-01), known counts and binary/empty data (TC-COMP-02)
Codec (`tests/test_codec.cpp`, Rachit)	UT-COD-01 to 07	7	Round trips: text, empty, binary, single byte; rejects bad metadata, trailing data, truncated single-symbol archive
Robustness (`tests/test_robustness.cpp`, Sprint 3)	UT-ROB-01 to 18	18	Missing/truncated headers, inconsistent metadata, bad bitstreams, unwritable outputs, overflow guards

---

5. Requirement traceability (updated RTM)

Requirement	System test cases	Unit test cases	Status
REQ-1 Accept a valid file path	TC-COMP-01, TC-ERR-01, TC-ERR-06, TC-ERR-07	UT-IN-01, UT-IN-02, UT-ROB-14	✅
REQ-2 Character frequencies	TC-COMP-02	UT-IN-03, UT-IN-04	✅
REQ-3 Huffman tree and bitstream	TC-COMP-03, TC-COMP-05	UT-HUF-01 to 07, UT-ROB-17, UT-ROB-18	✅
REQ-4 `.huff` output	TC-COMP-04, TC-COMP-07	UT-COD-01 to 04, UT-ROB-15	✅
REQ-5 Read metadata	TC-DEC-01, TC-ERR-08	UT-COD-05, UT-ROB-01 to 07	✅
REQ-6 Decode bitstream	TC-DEC-02, TC-ERR-03, TC-ERR-04	UT-COD-06, UT-COD-07, UT-ROB-08 to 13	✅
REQ-7 Lossless output file	TC-DEC-03, TC-DEC-04, TC-DEC-05, TC-NF-04	UT-COD-01 to 04, UT-ROB-16	✅
SRS 2.2 Metrics	TC-COMP-06	–	✅
SRS 6.1 Performance / memory	TC-NF-01, TC-NF-02	–	✅
SRS 6.2 Safety	TC-ERR-02 to 05, TC-ERR-08, TC-ERR-09, TC-SEC-01 (KL)	UT-ROB-01 to 16	✅ (D-16 known limitation)
SRS 2.4 / 6.4 Portability, correctness	TC-NF-03, TC-NF-04	all (CI on 3 OSes)	✅

Totals: 26 system and non-functional test cases (25 PASS, 1 KL) · 36 unit tests (36 PASS) · 17 lossless checks (17 PASS).